Compiler Design & Lexical Analysis

30 Lab Practice Problems & Solutions

01. Write a lex program to recognize valid Email Addresses.

```
%{
#include
%}
%%
[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,} { printf("Valid Email: %s\n",
yytext); }
. { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

02. Write a lex program to recognize valid IPv4 Addresses.

```
%{
#include
%}
%%
([0-9]{1,3}\.){3}[0-9]{1,3} { printf("Valid IPv4: %s\n", yytext); }
. { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

03. Write a lex program to identify valid URLs (http/https).

```
%{
#include
%}
%%
(http|https)://[a-zA-Z0-9.-]+\.[a-zA-Z]{2,} { printf("Valid URL: %s\n",
yytext); }
. { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

04. Write a lex program to recognize valid Date formats (DD/MM/YYYY).

```
%{
#include
%}
%%
(0[1-9]|[12][0-9]|3[01])[-/](0[1-9]|1[012])[-/](19|20)[0-9][0-9] { printf("Valid
Date: %s\n", yytext); }
. { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

05. Write a lex program to recognize Hexadecimal and Octal numbers.

```
%{
#include
%}
%%
0[xX][0-9a-fA-F]+ { printf("Hexadecimal: %s\n", yytext); }
0[0-7]+ { printf("Octal: %s\n", yytext); }
. { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

06. Write a lex program to count Vowels and Consonants in a file.

```
%{
int v=0, c=0;
%}
%%
[aeiouAEIOU] { v++; }
[a-zA-Z] { c++; }
. { /* ignore */ }
\n { /* ignore */ }
%%
int main() { yylex(); printf("Vowels: %d, Consonants: %d\n", v, c); return 0; }
int yywrap() { return 1; }
```

07. Write a lex program to count uppercase, lowercase letters, and digits.

```
%{
int up=0, low=0, dig=0;
%}
%%
[A-Z] { up++; }
[a-z] { low++; }
[0-9] { dig++; }
. { /* ignore */ }
%%
int main() { yylex(); printf("Upper: %d, Lower: %d, Digits: %d\n", up, low,
dig); return 0; }
int yywrap() { return 1; }
```

08. Write a lex program to count positive and negative numbers.

```
%{
int pos=0, neg=0;
%}
%%
[-][0-9]+(\.[0-9]+)? { neg++; }
[+]?[0-9]+(\.[0-9]+)? { pos++; }
. { /* ignore */ }
%%
int main() { yylex(); printf("Positive: %d, Negative: %d\n", pos, neg); return
0; }
int yywrap() { return 1; }
```

09. Write a lex program to remove blank lines from a text file.

```
%{
#include
%}
%%
^[ \t]*\n { /* Ignore blank lines */ }
. { ECHO; }
\n { ECHO; }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

10. Write a lex program to replace multiple spaces with a single space.

```
%{
#include
%}
%%
[ \t]+ { printf(" "); }
. { ECHO; }
\n { ECHO; }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

11. Write a lex program to count 'printf' and 'scanf' statements.

```
%{
int p=0, s=0;
%}
%%
"printf" { p++; }
"scanf" { s++; }
. { /* ignore */ }
\n { /* ignore */ }
%%
int main() { yylex(); printf("printf: %d, scanf: %d\n", p, s); return 0; }
int yywrap() { return 1; }
```

12. Write a lex program to recognize and count C keywords.

```
%{
int kw=0;
%}
%%
(int|float|char|double|if|else|while|for|return|void) { kw++; printf("Keyword:
%s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]* { /* Identifiers */ }
. { /* ignore */ }
%%
int main() { yylex(); printf("Total keywords: %d\n", kw); return 0; }
int yywrap() { return 1; }
```

13. Write a lex program to identify preprocessor directives.

```
%{
#include
%}
%%
^#include[ ]*<[^>]+> { printf("Include Directive: %s\n", yytext); }
^#define[ ]+[a-zA-Z_][a-zA-Z0-9_]*[ ]+.* { printf("Macro: %s\n", yytext); }
. { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

14. Write a lex program to extract string literals from a C program.

```
%{
#include
%}
%%
\"([^\\\"\n]|\\.)*\" { printf("String Literal: %s\n", yytext); }
. { /* ignore */ }
\n { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

15. Write a lex program to identify arithmetic and relational operators.

```
%{
#include
%}
%%
[\+\-\*/%] { printf("Arithmetic Operator: %s\n", yytext); }
(==|!=|<=|>=|<|>) { printf("Relational Operator: %s\n", yytext); }
. { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

16. Write a lex program to recognize articles (a, an, the).

```
%{
int count=0;
%}
%%
[ \t\n]([aA]|[aA][nN]|[tT][hH][eE])[ \t\n] { count++; printf("Article: %s\n",
yytext); }
. { /* ignore */ }
%%
int main() { yylex(); printf("Articles found: %d\n", count); return 0; }
int yywrap() { return 1; }
```

17. Write a lex program to recognize conjunctions (and, or, but).

```
%{
int count=0;
%}
%%
[ \t\n](and|or|but|AND|OR|BUT)[ \t\n] { count++; printf("Conjunction: %s\n",
yytext); }
. { /* ignore */ }
%%
int main() { yylex(); printf("Conjunctions: %d\n", count); return 0; }
int yywrap() { return 1; }
```

18. Write a lex program to convert lowercase letters to uppercase.

```
%{
#include
#include
%}
%%
[a-z] { printf("%c", toupper(yytext[0])); }
. { ECHO; }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

19. Write a lex program to check if a string is a palindrome.

```
%{
#include
#include
%}
%%
[a-zA-Z0-9]+ {
    int i, j, flag=1;
    for(i=0, j=yyleng-1; i
```

20. Write a lex program to accept strings starting with 'a' and ending with 'b'.

```
%{
#include
%}
%%
^a.*b$ { printf("Accepted: %s\n", yytext); }
.* { printf("Rejected: %s\n", yytext); }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

21. Write a program to eliminate Left Recursion from a CFG.

```
# Python logic for Left Recursion Elimination
productions = {'A': ['Aalpha', 'beta']}
new_grammar = {}
for non_term, rules in productions.items():
    alpha = [r[1:] for r in rules if r.startswith(non_term)]
    beta = [r for r in rules if not r.startswith(non_term)]
    if alpha:
        new_non_term = non_term + "'"
        new_grammar[non_term] = [b + new_non_term for b in beta]
        new_grammar[new_non_term] = [a + new_non_term for a in alpha] +
['epsilon']
print("Grammar after removing Left Recursion:", new_grammar)
```

22. Write a program to perform Left Factoring on a CFG.

```
# Python snippet for Left Factoring (A -> alpha beta1 | alpha beta2)
grammar = {'S': ['iEtS', 'iEtSeS', 'a']}
# Identifying common prefix 'iEtS'
new_grammar = {
    'S': ['iEtS S\'', 'a'],
    'S\'': ['eS', 'epsilon']
}
print("Left Factored Grammar:", new_grammar)
```

23. Write a program to calculate LEADING and TRAILING sets.

```
# Concept for calculating LEADING set
grammar = {'E': ['E+T', 'T'], 'T': ['T*F', 'F'], 'F': ['(E)', 'id']}
# LEADING(F) = {(, id}
# LEADING(T) = {*, (, id}
# LEADING(E) = {+, *, (, id}
print("LEADING(E) = {+, *, (, id}")
print("TRAILING(E) = {+, *, ), id}")
```

24. Write a program to construct LL(1) Parsing Table (Conceptual).

```
# Python Conceptual LL(1) Table construction
terminals = ['id', '+', '*', '(', ')', '$']
non_terminals = ['E', 'E\'']
table = {nt: {t: '' for t in terminals} for nt in non_terminals}
# Example entry based on FIRST set:
table['E']['id'] = 'E -> TE\''
table['E']['('] = 'E -> TE\''
print("LL(1) Parsing Table Entry: M[E, id] =", table['E']['id'])
```

25. Write a Recursive Descent Parser for E -> E+T | T.

```
/* C pseudocode for Recursive Descent */
void E() {
    T();
    Eprime();
}
void Eprime() {
    if (lookahead == '+') {
        match('+'); T(); Eprime();
    }
    /* else return (epsilon) */
}
void T() {
    F();
    Tprime();
}
```

26. Write a program to simulate a DFA accepting strings ending with 'abb'.

```
# Python DFA simulation
state = 'q0'
transitions = {
    'q0': {'a': 'q1', 'b': 'q0'},
    'q1': {'a': 'q1', 'b': 'q2'},
    'q2': {'a': 'q1', 'b': 'q3'},
    'q3': {'a': 'q1', 'b': 'q0'}
}
string = "ababb"
for char in string:
    state = transitions[state][char]
if state == 'q3':
    print("String Accepted")
else:
    print("String Rejected")
```

27. Write a lex program to identify and count HTML tags.

```
%{
int tags=0;
%}
%%
<[^>]+> { tags++; printf("HTML Tag: %s\n", yytext); }
. { /* ignore */ }
\n { /* ignore */ }
%%
int main() { yylex(); printf("Total tags: %d\n", tags); return 0; }
int yywrap() { return 1; }
```

28. Write a lex program to identify valid MAC Addresses.

```
%{
#include
%}
%%
([0-9A-Fa-f]{2}[:-]){5}([0-9A-Fa-f]{2}) { printf("MAC Address: %s\n", yytext); }
. { /* ignore */ }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

29. Write a program to compute CLOSURE of LR(0) items.

```
# Python conceptual CLOSURE for S' -> .S
def closure(items, grammar):
    closure_set = set(items)
    added = True
    while added:
        added = False
        for item in list(closure_set):
            dot_pos = item.find('.')
            if dot_pos != len(item) - 1:
                next_sym = item[dot_pos + 1]
                if next_sym in grammar:
                    for prod in grammar[next_sym]:
                        new_item = f"{next_sym}->.{prod}"
                        if new_item not in closure_set:
                            closure_set.add(new_item)
                            added = True
    return closure_set
```

30. Write a program to generate Three-Address Code (TAC) for a=b+c*d.

```
/* C conceptual logic for TAC Generation */
#include
int main() {
    printf("Input Expression: a = b + c * d\n");
    printf("Three Address Code:\n");
    printf("t1 = c * d\n");
    printf("t2 = b + t1\n");
    printf("a = t2\n");
    return 0;
}
```